

G.AJAY (192424297)

DSA0602-Data Handling And Visualization

Class Activity 2

Scenario 1: Smart City Air Quality Monitoring

PYTHON CODE:

```
# Import libraries
import sys
print(sys.executable)
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Generate sample dataset
np.random.seed(42)

n = 150

air_quality = pd.DataFrame({
    "Location": [f"L{i+1}" for i in range(n)],
    "PM2.5": np.random.randint(20, 180, n),
    "NO2": np.random.randint(10, 120, n),
    "CO": np.round(np.random.uniform(0.2, 3.0, n), 2),
    "Temperature": np.random.randint(22, 42, n),
    "Humidity": np.random.randint(40, 90, n)
})

# Display first few rows
print(air_quality.head())

# -----
# Visualization 1: Sequential Color Palette
# -----
plt.figure(figsize=(8,6))
sns.scatterplot(
    data=air_quality,
    x="Temperature",
    y="PM2.5",
    hue="PM2.5",
    palette="Reds",
    s=100
)
plt.title("PM2.5 Pollution (Sequential Color Scale)")
plt.xlabel("Temperature (°C)")
plt.ylabel("PM2.5")
```

```

plt.show()

# -----
# Visualization 2: Diverging Color Palette
# -----
plt.figure(figsize=(8,6))
sns.scatterplot(
    data=air_quality,
    x="Temperature",
    y="PM2.5",
    hue="PM2.5",
    palette="coolwarm",
    s=100
)
plt.title("PM2.5 Pollution (Diverging Color Scale)")
plt.xlabel("Temperature (°C)")
plt.ylabel("PM2.5")
plt.show()

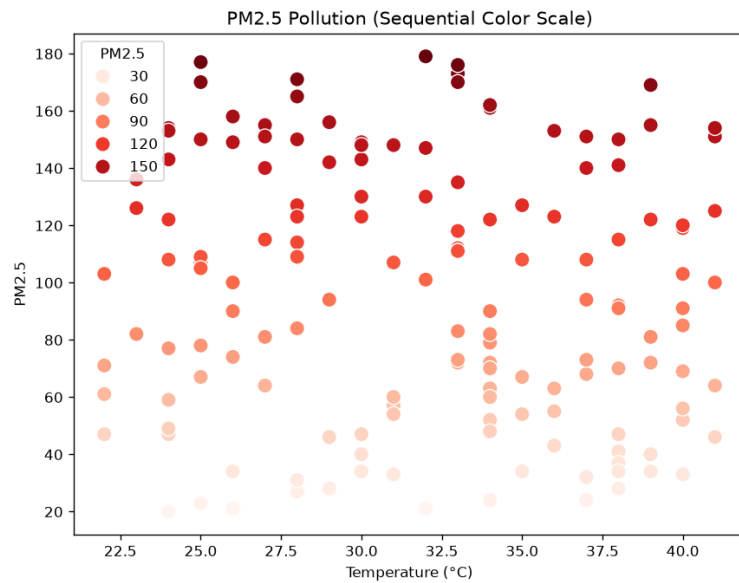
# -----
# Visualization 3: Qualitative Color Palette
# -----

# Create pollution categories
air_quality["Pollution_Level"] = pd.cut(
    air_quality["PM2.5"],
    bins=[0,50,100,150,200],
    labels=["Low", "Moderate", "High", "Very High"]
)
plt.figure(figsize=(8,6))
sns.scatterplot(
    data=air_quality,
    x="Temperature",
    y="PM2.5",
    hue="Pollution_Level",
    palette="Set2",
    s=100
)
plt.title("PM2.5 Pollution Categories (Qualitative Palette)")
plt.xlabel("Temperature (°C)")
plt.ylabel("PM2.5")
plt.show()

```

OUTPUT:

	Location	PM2.5	NO2	CO	Temperature	Humidity
0	L1	122	86	2.58	34	63
1	L2	112	114	2.04	33	67
2	L3	34	12	1.79	35	47
3	L4	126	112	0.46	23	67
4	L5	91	79	1.23	40	75



Scenario 2: Earthquake Risk Analysis

PYTHON CODE:

```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Create sample dataset
np.random.seed(42)
n = 200
earthquake = pd.DataFrame({
    "Magnitude": np.round(np.random.uniform(0.5, 9.5, n), 1),
    "Depth": np.random.randint(1, 701, n),
    "Latitude": np.round(np.random.uniform(-90, 90, n), 2),
    "Longitude": np.round(np.random.uniform(-180, 180, n), 2),
    "AftershockFrequency": np.random.randint(1, 500, n)
})

# Display first few rows
print(earthquake.head())

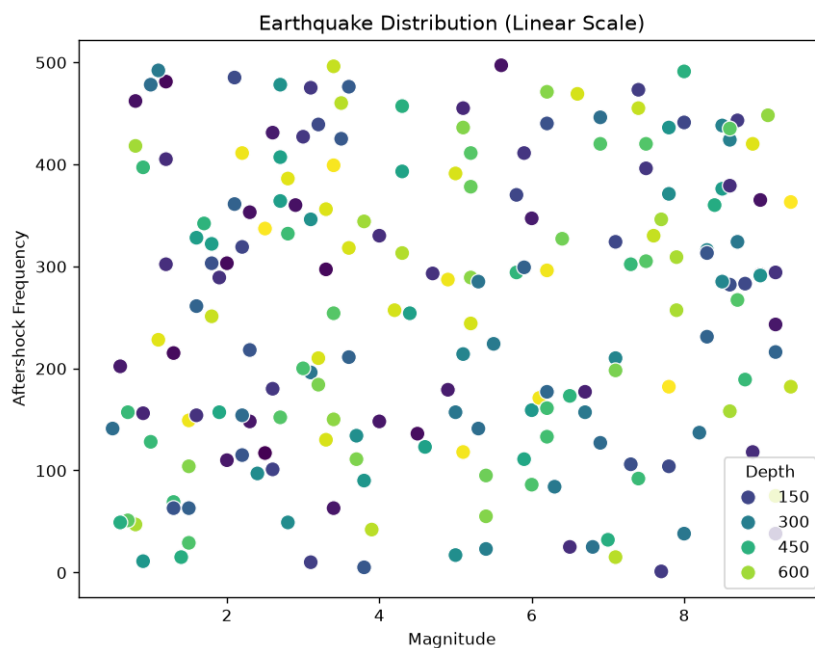
plt.figure(figsize=(8,6))
sns.scatterplot(
    data=earthquake,
    x="Magnitude",
    y="AftershockFrequency",
    hue="Depth",
    palette="viridis",
    s=80
)
```

```
plt.title("Earthquake Distribution (Linear Scale)")
plt.xlabel("Magnitude")
plt.ylabel("Aftershock Frequency")
plt.show()

plt.figure(figsize=(8,6))
sns.scatterplot(
    data=earthquake,
    x="Magnitude",
    y="AftershockFrequency",
    hue="Depth",
    palette="viridis",
    s=80
)
plt.yscale("log")
plt.title("Earthquake Distribution (Logarithmic Scale)")
plt.xlabel("Magnitude")
plt.ylabel("Aftershock Frequency (Log Scale)")
plt.show()
```

OUTPUT:

	Magnitude	Depth	Latitude	Longitude	AftershockFrequency
0	3.9	628	20.85	-148.17	42
1	9.1	587	79.90	105.05	448
2	7.1	625	79.97	32.38	15
3	5.9	420	66.10	-7.18	111
4	1.9	422	24.55	-28.61	157



Scenario 3: Hospital Patient Admission Analysis

PYTHON CODE:

```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Create sample dataset
np.random.seed(42)
n = 365
wind = pd.DataFrame({
    "WindSpeed": np.random.randint(5, 40, n),      # km/h
    "WindDirection": np.random.randint(0, 360, n)  # degrees
})

print(wind.head())

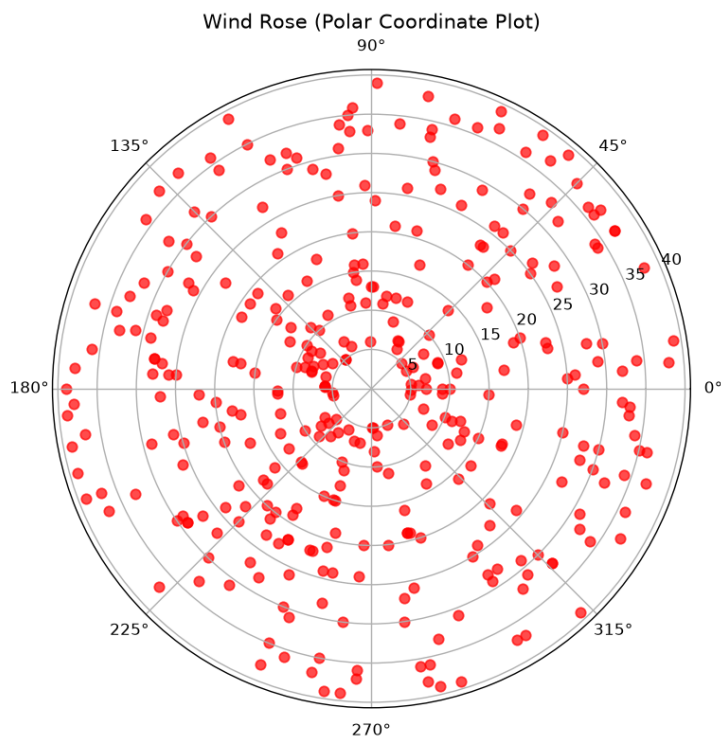
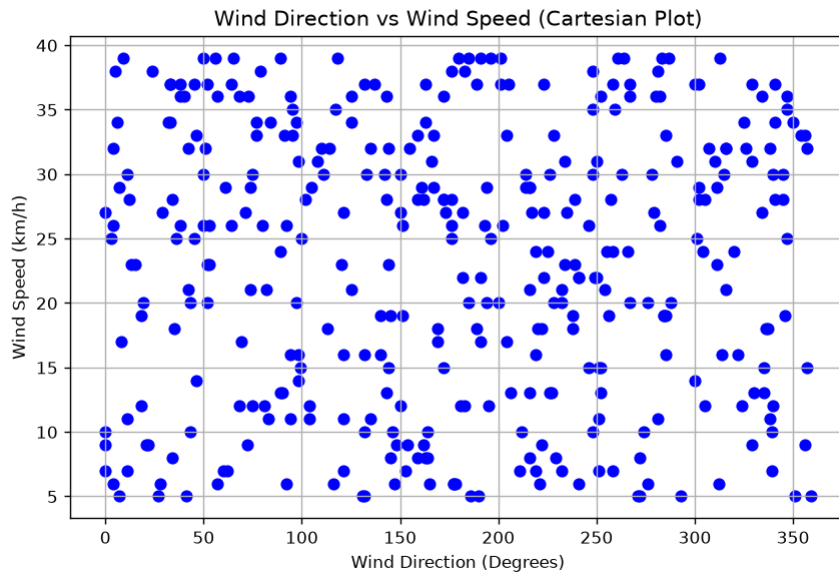
plt.figure(figsize=(8,5))
plt.scatter(
    wind["WindDirection"],
    wind["WindSpeed"],
    color="blue"
)

plt.title("Wind Direction vs Wind Speed (Cartesian Plot)")
plt.xlabel("Wind Direction (Degrees)")
plt.ylabel("Wind Speed (km/h)")
plt.grid(True)
plt.show()

theta = np.deg2rad(wind["WindDirection"])
plt.figure(figsize=(7,7))
ax = plt.subplot(111, projection='polar')
ax.scatter(
    theta,
    wind["WindSpeed"],
    color="red",
    alpha=0.7
)
ax.set_title("Wind Rose (Polar Coordinate Plot)")
plt.show()
```

OUTPUT:

	WindSpeed	WindDirection
0	33	95
1	19	256
2	12	324
3	25	3
4	23	15



Scenario 4: Hospital Patient Monitoring Dashboard

PYTHON CODE:

```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Create sample dataset
np.random.seed(42)
n = 20
```

```

patients = pd.DataFrame({
    "HeartRate": np.random.randint(50, 130, n),          # bpm
    "Oxygen": np.random.randint(85, 100, n),            # %
    "BloodPressure": np.random.randint(90, 170, n),     # mmHg
    "Respiration": np.random.randint(10, 30, n),        # breaths/min
    "Temperature": np.round(np.random.uniform(35, 40, n),1) # °C
})

print(patients)

# Heart Rate Dashboard
for hr in patients["HeartRate"]:
    if hr < 60 or hr > 100:
        colors.append("red")          # Abnormal
    elif 60 <= hr <= 100:
        colors.append("green")        # Normal

plt.figure(figsize=(10,5))
plt.bar(range(n), patients["HeartRate"], color=colors)

plt.title("ICU Patient Heart Rate Dashboard")
plt.xlabel("Patient Number")
plt.ylabel("Heart Rate (bpm)")
plt.axhline(60, color='blue', linestyle='--')
plt.axhline(100, color='blue', linestyle='--')
plt.show()

# Oxygen Saturation Dashboard
colors = []

for spo2 in patients["Oxygen"]:
    if spo2 < 95:
        colors.append("red")
    else:
        colors.append("green")

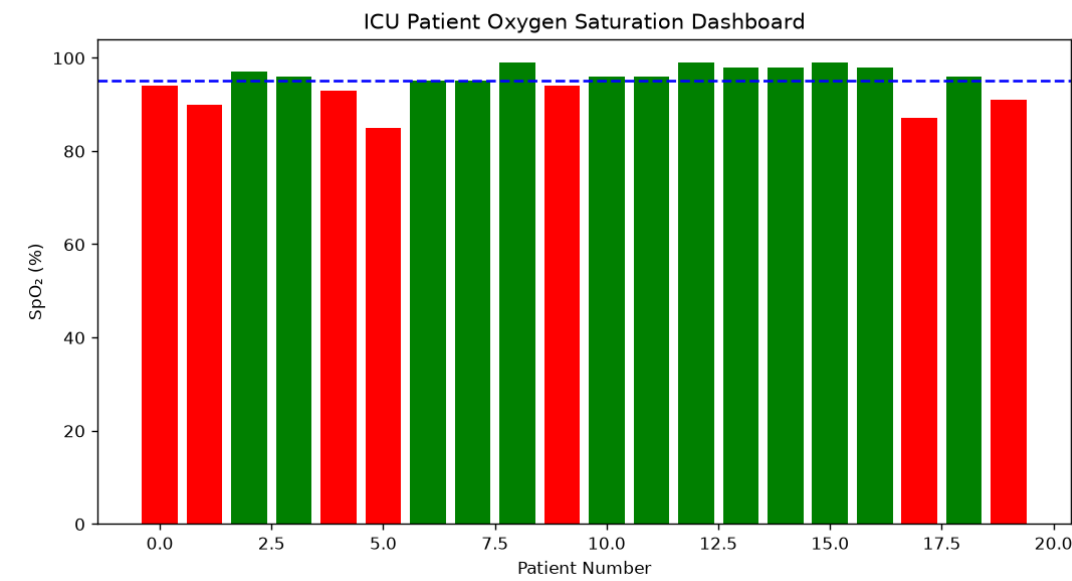
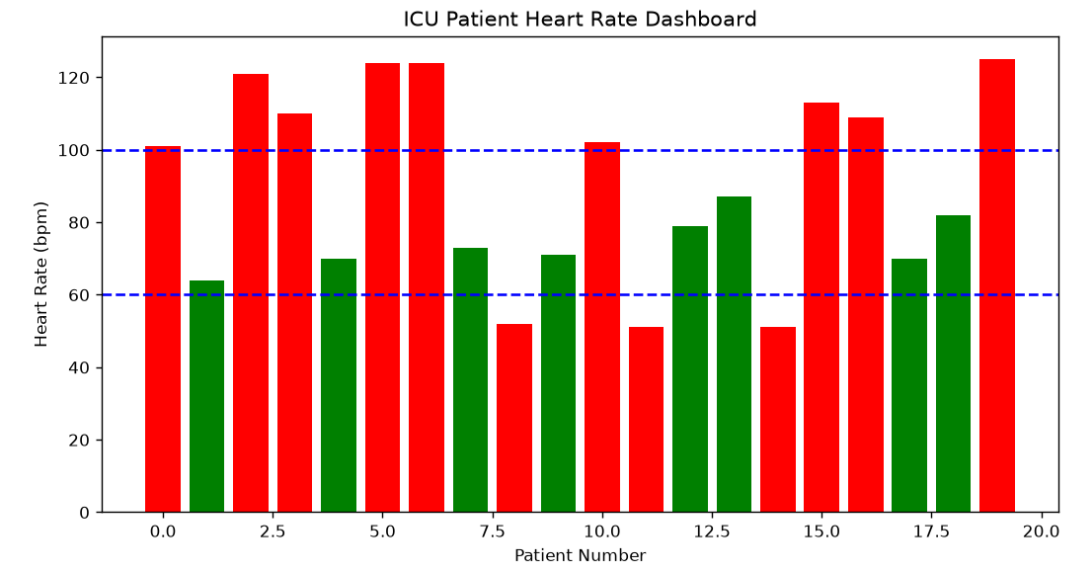
plt.figure(figsize=(10,5))
plt.bar(range(n), patients["Oxygen"], color=colors)

plt.title("ICU Patient Oxygen Saturation Dashboard")
plt.xlabel("Patient Number")
plt.ylabel("SpO2 (%)")
plt.axhline(95, color='blue', linestyle='--')
plt.show()

```

OUTPUT:

	HeartRate	Oxygen	BloodPressure	Respiration	Temperature
0	101	94	153	23	36.0
1	64	90	92	26	35.2
2	121	97	140	13	36.6
3	110	96	96	27	36.9
4	70	93	110	17	36.4
5	124	85	162	13	39.1
6	124	95	128	11	36.8
7	73	95	107	15	36.4
8	52	99	93	19	37.7
9	71	94	149	13	35.7
10	102	96	103	27	39.0
11	51	96	98	21	35.4
12	79	99	142	11	39.9
13	87	98	91	19	38.9
14	51	98	149	13	36.0
15	113	99	160	23	35.0
16	109	98	133	25	39.1
17	70	87	97	24	38.5
18	82	96	136	17	38.6
19	125	91	124	23	38.9



Scenario 5: Global Climate Change Study

PYTHON CODE:

```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Create sample dataset
np.random.seed(42)

countries = ["India", "USA", "China", "Brazil", "Australia",
             "Canada", "Germany", "Japan", "Russia", "South Africa"]

years = [1980, 1990, 2000, 2010, 2020, 2025]

# Generate temperature anomaly data
data = np.random.uniform(-2.5, 2.5, (len(countries), len(years)))

climate = pd.DataFrame(data, index=countries, columns=years)

print(climate)

# -----
# Heatmap 1 : Coolwarm Palette
# -----

plt.figure(figsize=(8,6))
sns.heatmap(climate,
            cmap="coolwarm",
            annot=True,
            center=0)

plt.title("Temperature Anomalies - Coolwarm")
plt.xlabel("Year")
plt.ylabel("Country")
plt.show()

# -----
# Heatmap 2 : RdBu Palette
# -----

plt.figure(figsize=(8,6))
sns.heatmap(climate,
            cmap="RdBu",
            annot=True,
            center=0)
```

```
plt.title("Temperature Anomalies - RdBu")
plt.xlabel("Year")
plt.ylabel("Country")
plt.show()
```

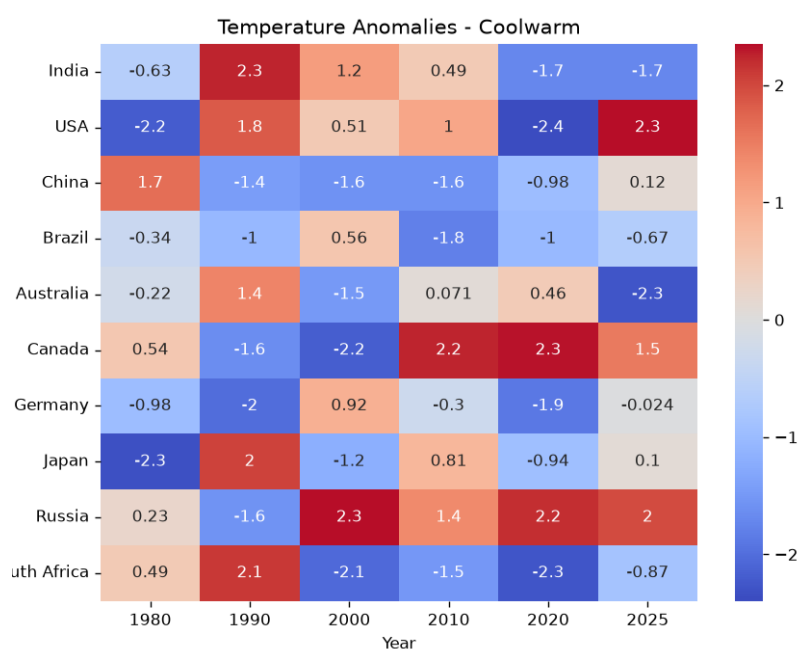
```
# -----
# Heatmap 3 : PiYG Palette
# -----
```

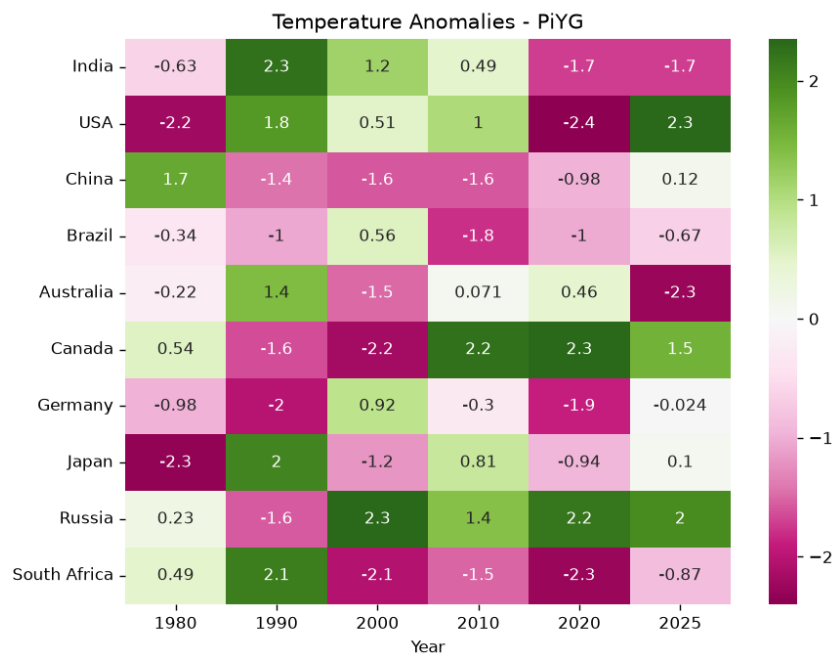
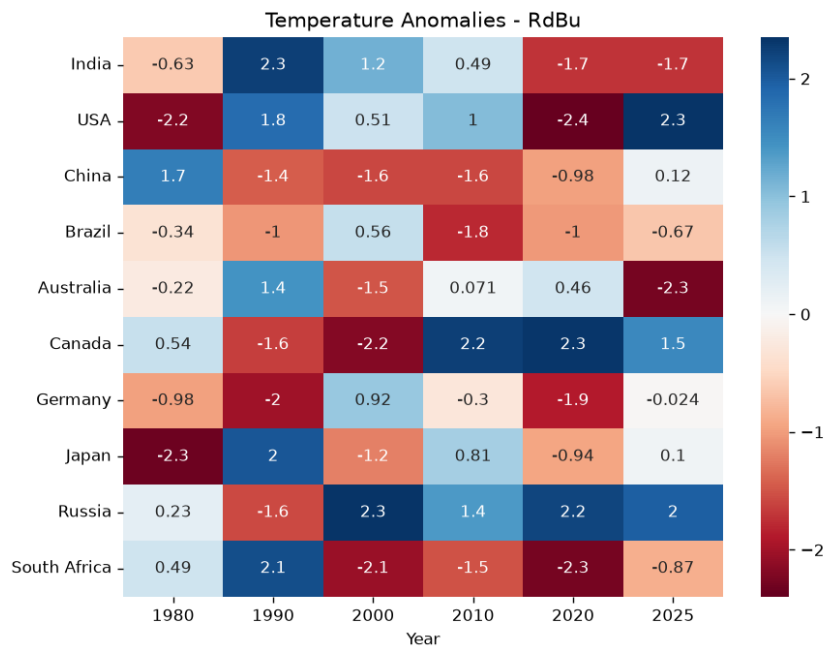
```
plt.figure(figsize=(8,6))
sns.heatmap(climate,
            cmap="PiYG",
            annot=True,
            center=0)
```

```
plt.title("Temperature Anomalies - PiYG")
plt.xlabel("Year")
plt.ylabel("Country")
plt.show()
```

OUTPUT:

	1980	1990	2000	2010	2020	2025
India	-0.627299	2.253572	1.159970	0.493292	-1.719907	-1.720027
USA	-2.209582	1.830881	0.505575	1.040363	-2.397078	2.349549
China	1.662213	-1.438304	-1.590875	-1.582977	-0.978789	0.123782
Brazil	-0.340275	-1.043854	0.559264	-1.802531	-1.039277	-0.668191
Australia	-0.219650	1.425880	-1.501631	0.071172	0.462073	-2.267748
Canada	0.537724	-1.647379	-2.174742	2.244428	2.328160	1.541987
Germany	-0.976931	-2.011639	0.921165	-0.299238	-1.889809	-0.024115
Japan	-2.328057	2.046602	-1.206100	0.812611	-0.941445	0.100340
Russia	0.233551	-1.575728	2.347923	1.375664	2.197495	1.974137
South Africa	0.489500	2.109371	-2.057537	-1.520086	-2.273864	-0.873348





Scenario 6: E-Commerce Sales Analytics

PYTHON CODE:

```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Create sample dataset
np.random.seed(42)
```

```

categories = ["Electronics", "Clothing", "Furniture", "Groceries",
              "Books", "Sports", "Toys", "Beauty"]

regions = ["North", "South", "East", "West"]

data = []

for category in categories:
    for region in regions:
        sales = np.random.randint(1000, 10000)
        data.append([category, region, sales])

sales_data = pd.DataFrame(data, columns=["Category", "Region", "Sales"])

print(sales_data)

# -----
# Visualization 1 : Bar Chart
# Category = Color (Qualitative)
# Sales = Height (Quantitative)
# -----

plt.figure(figsize=(10,6))

sns.barplot(
    data=sales_data,
    x="Category",
    y="Sales",
    hue="Category",
    palette="Set2",
    legend=False
)

plt.title("Sales by Product Category")
plt.xticks(rotation=45)
plt.show()

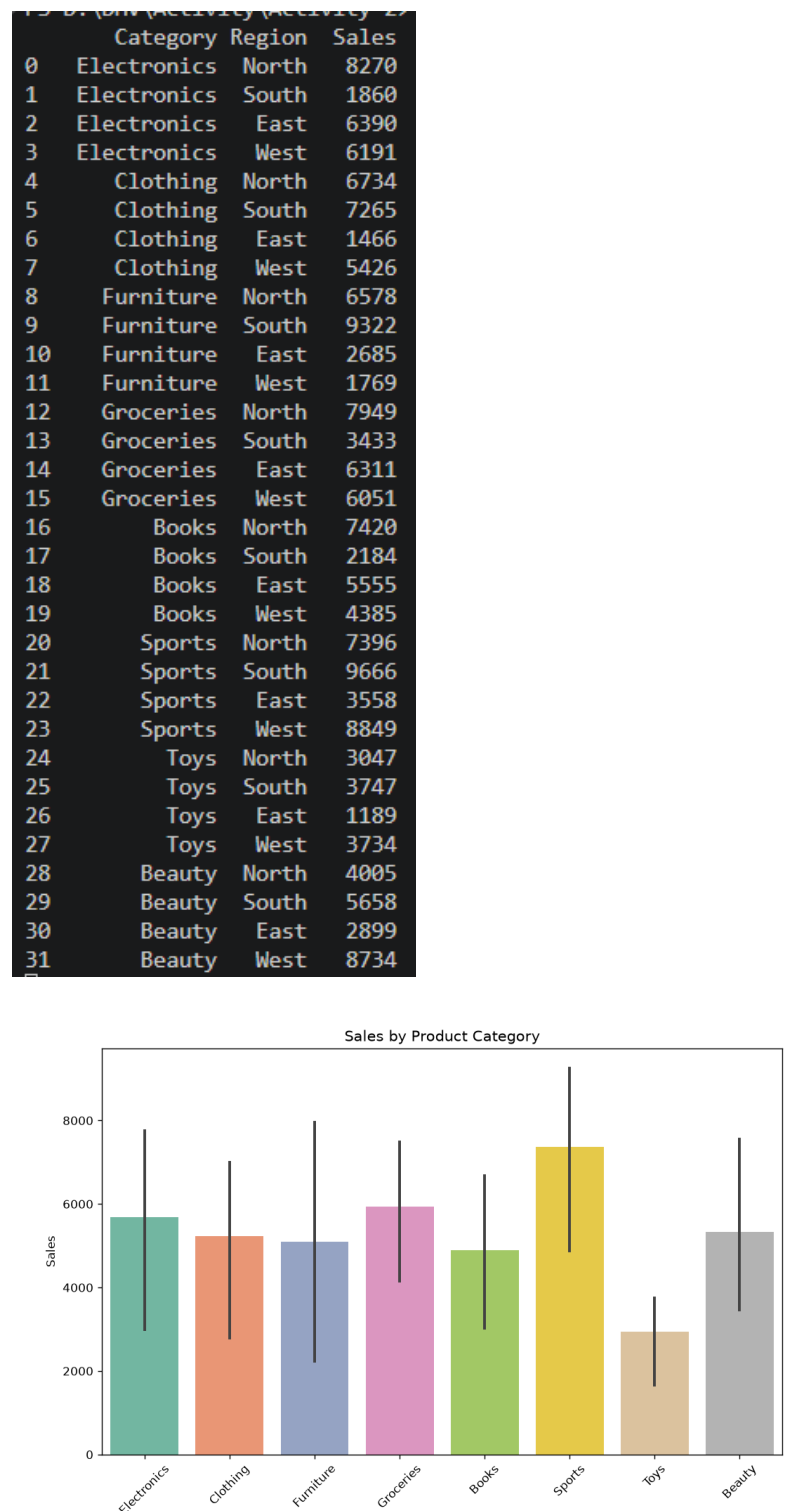
# Visualization 2 : Scatter Plot
# Category = Color
# Sales = Point Size
plt.figure(figsize=(10,6))

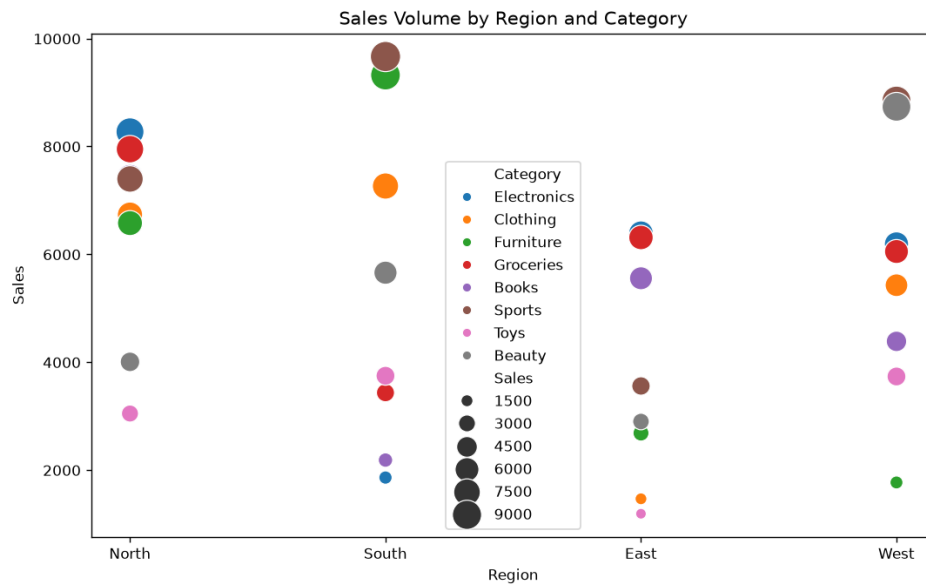
sns.scatterplot(
    data=sales_data,
    x="Region",
    y="Sales",
    hue="Category",
    size="Sales",

```

```
        sizes=(50,400),
        palette="tab10"
    )
plt.title("Sales Volume by Region and Category")
plt.show()
```

OUTPUT:





Scenario 7: Satellite Image Vegetation Analysis

PYTHON CODE:

```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Create sample dataset
np.random.seed(42)

rows = 10
cols = 10

# NDVI values range from -1 to +1
ndvi = np.random.uniform(-1, 1, (rows, cols))

print(ndvi)

# -----
# Visualization 1 : Sequential Palette
# -----

plt.figure(figsize=(6,5))

sns.heatmap(
    ndvi,
    cmap="Greens",
    cbar=True
)
```

```
plt.title("NDVI Heatmap - Sequential Palette (Greens)")
plt.show()

# -----
# Visualization 2 : Diverging Palette (RdYlGn)
# -----

plt.figure(figsize=(6,5))

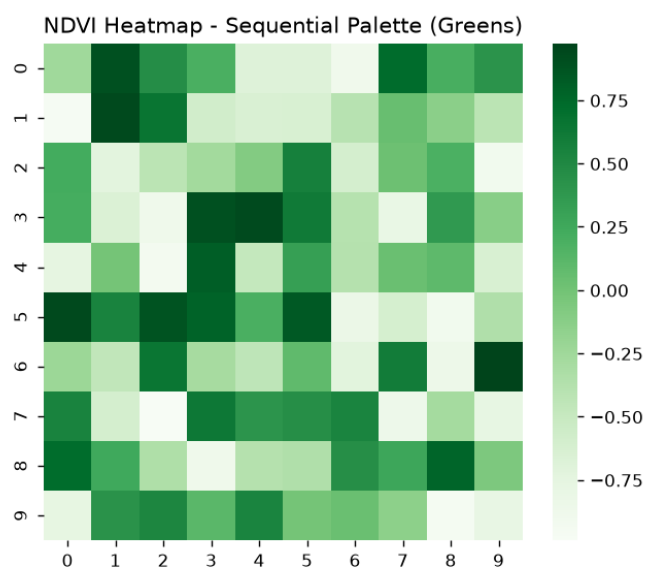
sns.heatmap(
    ndvi,
    cmap="RdYlGn",
    center=0,
    cbar=True
)

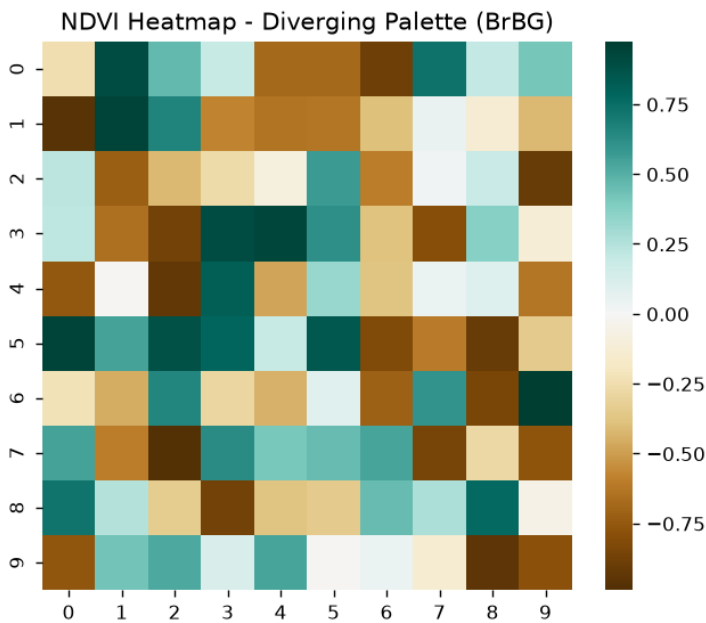
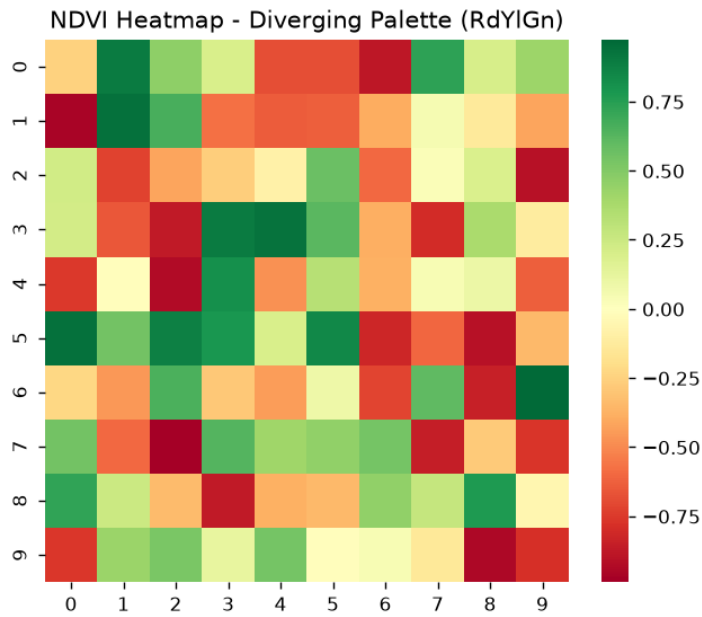
plt.title("NDVI Heatmap - Diverging Palette (RdYlGn)")
plt.show()
# Visualization 3 : Diverging Palette (BrBG)
plt.figure(figsize=(6,5))

sns.heatmap(
    ndvi,
    cmap="BrBG",
    center=0,
    cbar=True
)

plt.title("NDVI Heatmap - Diverging Palette (BrBG)")
plt.show()
```

OUTPUT:





Scenario 8: Cryptocurrency Market Volatility

PYTHON CODE:

```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Create sample dataset
np.random.seed(42)

days = np.arange(1, 101)
```

```

crypto = pd.DataFrame({
    "Day": days,
    "Bitcoin": np.random.randint(30000, 70000, 100),
    "Ethereum": np.random.randint(1000, 5000, 100),
    "Solana": np.random.randint(20, 250, 100),
    "Cardano": np.round(np.random.uniform(0.2, 2.5, 100), 2)
})

print(crypto.head())

# -----
# Visualization 1 : Cartesian Line Plot
# -----

plt.figure(figsize=(10,6))

plt.plot(crypto["Day"], crypto["Bitcoin"], label="Bitcoin", color="orange")
plt.plot(crypto["Day"], crypto["Ethereum"], label="Ethereum", color="blue")
plt.plot(crypto["Day"], crypto["Solana"], label="Solana", color="green")
plt.plot(crypto["Day"], crypto["Cardano"], label="Cardano", color="red")

plt.title("Cryptocurrency Prices (Linear Scale)")
plt.xlabel("Day")
plt.ylabel("Price (USD)")
plt.legend()
plt.grid(True)
plt.show()

# -----
# Visualization 2 : Logarithmic Scale
# -----

plt.figure(figsize=(10,6))

plt.plot(crypto["Day"], crypto["Bitcoin"], label="Bitcoin", color="orange")
plt.plot(crypto["Day"], crypto["Ethereum"], label="Ethereum", color="blue")
plt.plot(crypto["Day"], crypto["Solana"], label="Solana", color="green")
plt.plot(crypto["Day"], crypto["Cardano"], label="Cardano", color="red")

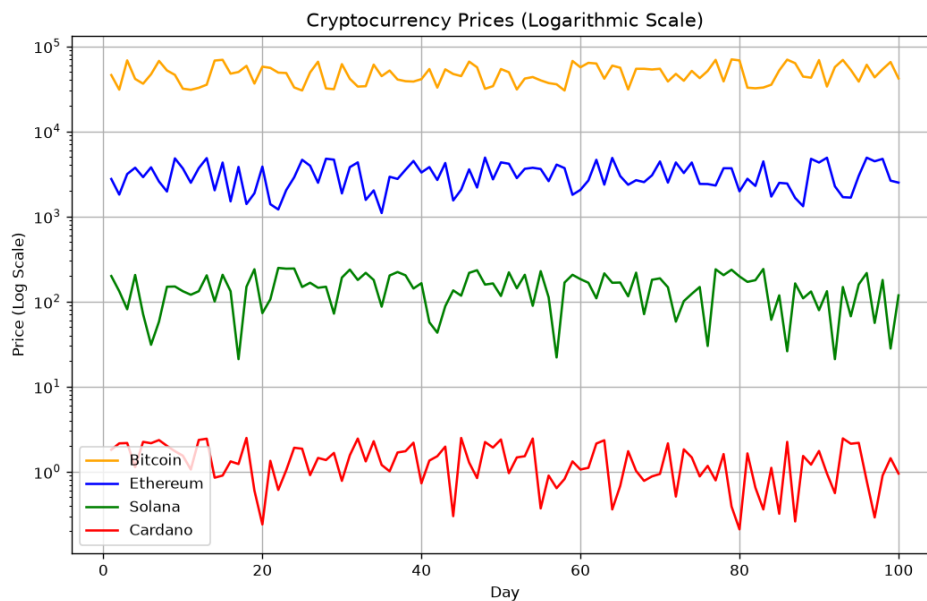
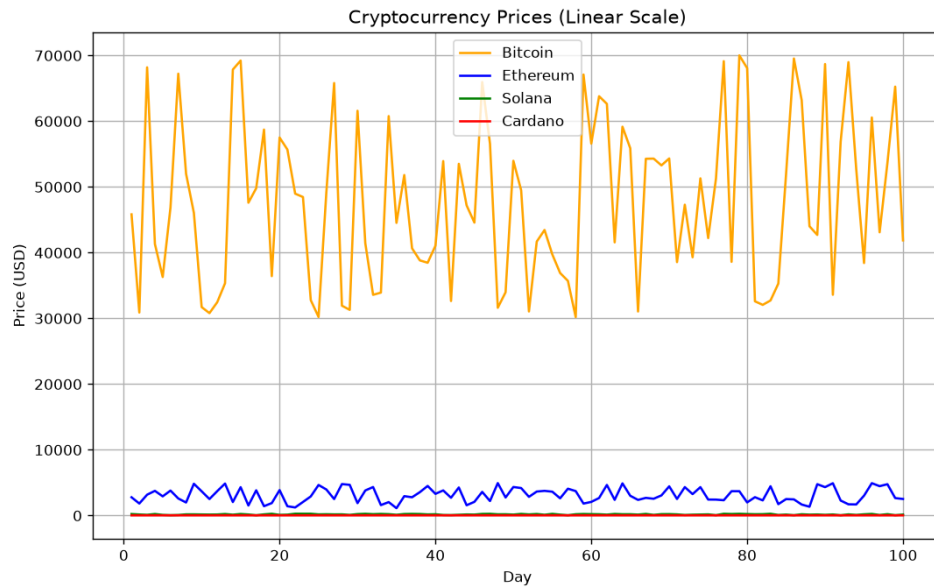
plt.yscale("log")

plt.title("Cryptocurrency Prices (Logarithmic Scale)")
plt.xlabel("Day")
plt.ylabel("Price (Log Scale)")
plt.legend()
plt.grid(True)
plt.show()

```

OUTPUT:

	Day	Bitcoin	Ethereum	Solana	Cardano
0	1	45795	2751	199	1.81
1	2	30860	1804	132	2.15
2	3	68158	3146	81	2.17
3	4	41284	3731	205	1.13
4	5	36265	2895	71	2.24



Scenario 9: Marathon Runner Performance Analysis

PYTHON CODE:

```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Create sample dataset
```

```

np.random.seed(42)

n = 150

runner = pd.DataFrame({
    "Distance": np.linspace(0, 42.2, n),          # km
    "Speed": np.random.uniform(8, 18, n),         # km/h
    "HeartRate": np.random.randint(110, 190, n),  # bpm
    "Elevation": np.random.randint(0, 500, n),    # meters
    "Direction": np.random.randint(0, 360, n)     # degrees
})

print(runner.head())

# -----
# Visualization 1 : Cartesian Scatter Plot
# -----

plt.figure(figsize=(10,6))

plt.scatter(
    runner["Distance"],
    runner["Elevation"],
    c=runner["Speed"],
    cmap="viridis",
    s=60
)

plt.colorbar(label="Speed (km/h)")
plt.title("Runner Performance (Cartesian Plot)")
plt.xlabel("Distance (km)")
plt.ylabel("Elevation (m)")
plt.grid(True)
plt.show()

# -----
# Visualization 2 : Polar Coordinate Plot
# -----

theta = np.deg2rad(runner["Direction"])

plt.figure(figsize=(7,7))
ax = plt.subplot(111, projection="polar")

scatter = ax.scatter(
    theta,
    runner["Distance"],
    c=runner["Speed"],

```

```

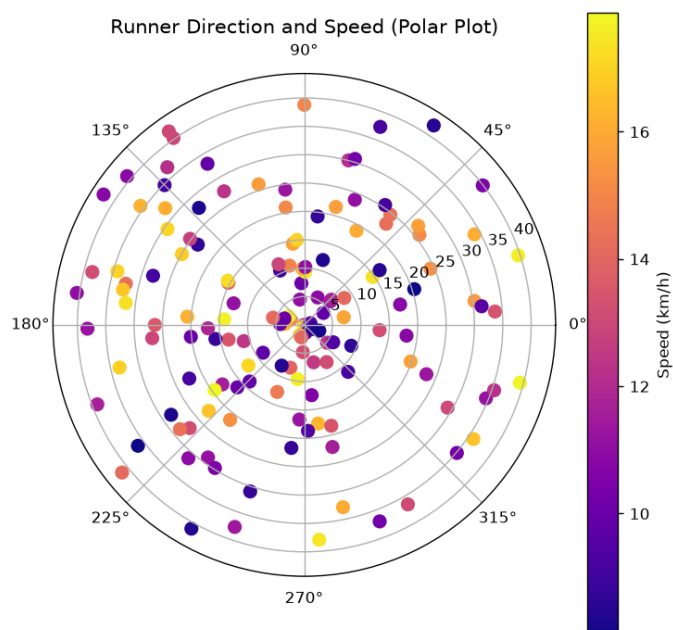
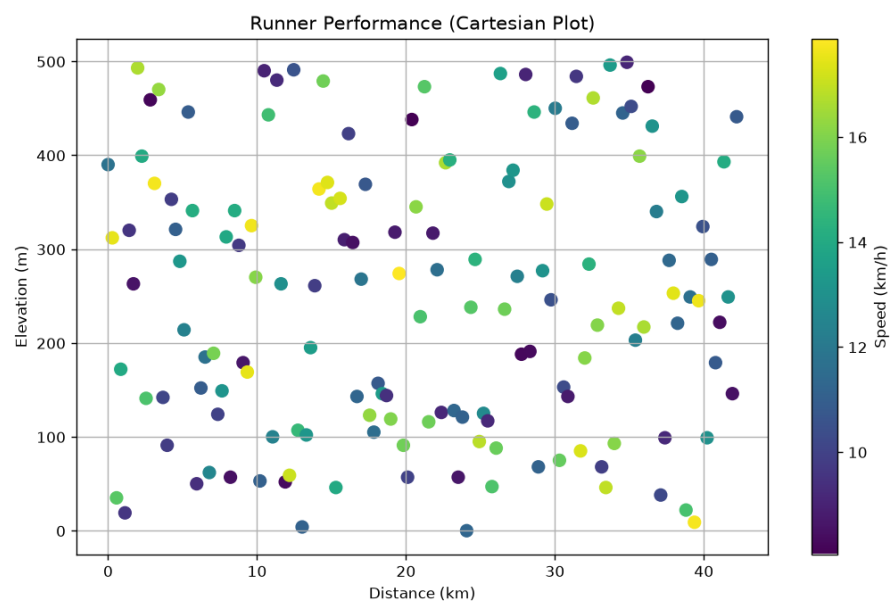
    cmap="plasma",
    s=60
)

plt.colorbar(scatter, label="Speed (km/h)")
ax.set_title("Runner Direction and Speed (Polar Plot)")
plt.show()

```

OUTPUT:

	Distance	Speed	HeartRate	Elevation	Direction
0	0.000000	11.745401	110	390	95
1	0.283221	17.507143	128	312	256
2	0.566443	15.319939	111	35	324
3	0.849664	13.986585	162	172	3
4	1.132886	9.560186	153	19	15



Scenario 10: COVID-19 Vaccination Campaign Dashboard

PYTHON CODE:

```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Create sample dataset
np.random.seed(42)

vaccination = pd.DataFrame({
    "District": ["A", "B", "C", "D", "E", "F", "G", "H"],
    "VaccinationRate": np.random.randint(55, 100, 8),
    "AgeGroup": ["18-30", "31-45", "46-60", "60+", "18-30", "31-45", "46-60", "60+"],
    "VaccineType": ["Covishield", "Covaxin", "Covishield", "Covaxin",
                    "Covishield", "Covaxin", "Covishield", "Covaxin"]
})

print(vaccination)

# -----
# Visualization 1 : Policymakers Dashboard
# Sequential Color Palette
# -----

plt.figure(figsize=(8,5))

sns.barplot(
    data=vaccination,
    x="District",
    y="VaccinationRate",
    hue="VaccineType",
    palette="Blues"
)

plt.title("COVID-19 Vaccination Rates (Policymakers)")
plt.xlabel("District")
plt.ylabel("Vaccination Rate (%)")
plt.show()

plt.figure(figsize=(8,5))
sns.barplot(
    data=vaccination,
    x="District",
    y="VaccinationRate",
    hue="VaccineType",
    palette="colorblind"
)
```

```
# Highlight districts with vaccination below 70%
for i, value in enumerate(vaccination["VaccinationRate"]):
    if value < 70:
        plt.text(i, value + 1, "Low", ha="center", color="red", fontsize=10)
plt.title("COVID-19 Vaccination Rates (General Public)")
plt.xlabel("District")
plt.ylabel("Vaccination Rate (%)")
plt.show()
```

OUTPUT:

	District	VaccinationRate	AgeGroup	VaccineType
0	A	93	18-30	Covishield
1	B	83	31-45	Covaxin
2	C	69	46-60	Covishield
3	D	97	60+	Covaxin
4	E	62	18-30	Covishield
5	F	75	31-45	Covaxin
6	G	93	46-60	Covishield
7	H	73	60+	Covaxin

